

DRISHTI-AI: Design Brief — Built for Sustainability, Availability, Accessibility, and Platform Independence

From: Bezaleel (Team Lead / Full-Stack Architect) **To:** Madhu, Akshay, Adithya **Subject:** Why We're Building This Way, and What Each of Us Owns **Date:** 7 Sep 2026 (Day 1)

STARTING POINT

India has 77+ million diabetic adults. Diabetic retinopathy is the leading cause of preventable blindness among them, and rural India has roughly one ophthalmologist per 100,000 people. AI models capable of detecting DR from a retinal image already exist and have been validated — Google's ARDA system (piloted with hospital partners including Aravind Eye Hospital) and commercial tools like IDx-DR and EyeArt have proven the underlying approach works clinically.

What hasn't been solved is getting a system like this to actually run, day after day, in a rural PHC with unreliable power, patchy connectivity, a ₹15,000 camera, a non-specialist operator, and no guarantee of a fixed device or OS. That's the gap DRISHTI-AI is built to close. Every design decision below is made against four questions, in this order: **Will it keep running here? Can everyone reach it? Can everyone use it? Will it run on whatever hardware they actually have?**

THE FOUR DESIGN PILLARS

1. Sustainability

The system is built to run indefinitely on modest, low-power hardware rather than depending on continuous cloud compute:

- Inference runs on-device/on-edge wherever possible, so a PHC isn't paying an ongoing cloud compute or data bill just to keep screening patients.
- Edge-filtering cuts data volume by ~98.6% (250GB raw → 3.4GB) before anything needs to leave the site, reducing both bandwidth cost and the energy footprint of transmission.
- No dependency on frequent hardware refreshes — the pipeline is designed to keep working on the same low-cost camera and compute setup for years, rather than requiring newer devices as the "official" supported path narrows.
- Quality-gating itself is a sustainability decision: rejecting a bad image before it reaches the classifier avoids wasted compute on unusable input, and avoids the downstream cost (a wrong referral, a missed case, a repeat visit) of a forced bad prediction.

2. Availability

The system must keep working when connectivity, power, or specialist staff don't:

- Full on-site screening and reporting work with no live internet connection; sync to central systems happens opportunistically when a connection is available.
- Designed for 2G/3G-viable data

volumes, not broadband-only. - Discrete-event simulation across 100,000 patients/year, 50 PHCs, and 1 district hospital shows the model holds up at real operating scale, not just in a demo. - Reduces the specialist bottleneck directly — cutting required ophthalmologist review capacity from ~13 to ~2 tele-reviewers per 100k patients — so availability isn't gated on hiring specialists rural facilities can't get.

3. Accessibility

The system must be usable by the people actually present at a rural PHC, not just by a specialist reading a dashboard: - Operated by a trained technician, not a doctor — quality-gate feedback and recapture prompts are written in plain, actionable language. - Patient-facing reports are bilingual (English + Hindi) today, with additional regional languages (Tamil, Telugu, Kannada) scoped as a near-term addition, since Hindi alone doesn't cover the highest-DR-burden regions of South India. - Works with low-cost, widely available fundus cameras (~₹15,000) instead of requiring hospital-grade imaging equipment. - Output is designed to be understandable at a glance — clear severity indication alongside the technical grade, not just a number a technician has to interpret.

4. Platform Independence

The system must not lock a health system into one vendor, one OS, or one cloud provider: - Built on an open-source stack, released as open-source software, so it can be inspected, adapted, and run without a licensing gate. - No hard dependency on a single cloud provider — the pipeline is designed to run locally first, with cloud/central sync as an optional layer, not a requirement. - Standards-based interoperability (FHIR R4, SNOMED CT, LOINC, ABDM/Ayushman Bharat alignment) means the data isn't trapped in a proprietary format — any compliant health system tooling can consume it. - Hardware-agnostic on the imaging side — not tied to one camera manufacturer's SDK or format.

THE WORKFLOW, THROUGH THOSE FOUR LENSES

Patient arrives at PHC

↓

[Clinical Risk Engine] — upstream diabetes risk check (ICMR 2024 thresholds,

lab biomarkers where available) — routes clearly high-risk patients to

lab testing instead of unnecessary imaging → AVAILABILITY (don't waste

scarce imaging time), SUSTAINABILITY (don't run inference that isn't needed)

↓

Technician captures retinal image on low-cost fundus camera

→ ACCESSIBILITY (any trained technician, any compatible low-cost camera),

PLATFORM INDEPENDENCE (not locked to one camera vendor)

↓

[Image Quality Gate] — Good / Borderline / Bad, evaluated on-device

→ SUSTAINABILITY (no wasted downstream compute on bad input),

AVAILABILITY (works with no connection, since it's on-device)

Bad → recapture prompt in plain language, max 2 retries → human review

→ ACCESSIBILITY

↓

[DR Severity Classifier] — runs only on images that passed the gate,

on-device, no cloud round-trip required
→ AVAILABILITY, SUSTAINABILITY
↓
[Grad-CAM Explainability] – visual heatmap in ~1.14s, so a technician or reviewing clinician can see *why*, not just *what*
→ ACCESSIBILITY
↓
[Confidence Check] – low-confidence or high-risk (Grade 3/4) always flagged for human review, never resolved by the model alone
→ built-in safety, supports future regulatory compliance path
↓
[Retinal Structure Segmentation] – optic disc, fovea, vessel tree, microaneurysm/exudate candidates, CSME risk – combined overlay for <30s specialist review
→ AVAILABILITY (makes remote/tele-review fast enough to scale)
↓
[Clinical Reporting] – bilingual PDF report + FHIR R4/ABDM export
→ ACCESSIBILITY (patient can read it), PLATFORM INDEPENDENCE (any standards-compliant system can ingest it)
↓
Edge-filtered data (98.6% smaller) syncs to district/central system when connectivity allows – never blocks on-site screening
→ AVAILABILITY, SUSTAINABILITY (less data transmitted, less energy used)

WHAT THIS ISN'T CLAIMING

- We're not claiming to have invented image-quality gating as a concept — it's an established, expected part of serious clinical DR-screening AI (ARDA, IDx-DR, EyeArt all do it). We're building an independent, open implementation of it, integrated into a fuller system aimed at rural conditions specifically.
 - We're not claiming clinical certification. A real deployment would need CDSCO Medical Device Software approval (likely Class B/C under India's 2026 guidance) and DPDP Act-compliant handling of real patient data. The human-in-the-loop, abstain-and-escalate design is built with that path in mind, not to bypass it.
 - We're not claiming diagnostic accuracy beyond what's validated. Confidence is reported honestly, Grad-CAM is explicitly attention visualization and not lesion segmentation, and every high-risk or low-confidence case goes to a person.
-

WHAT'S ALREADY BUILT

Module	Status	Key File
Image Quality Gate	DONE	src/quality/checker.py, src/quality/enhancer.py
DR Severity Classifier	DONE + trained model	src/classification/classifier.py, final_model.keras (33.4 MB)
Grad-CAM Explainability	DONE	src/classification/gradcam.py
Pipeline Router	DONE — full 11-node decision flow	src/pipeline/router.py
Confidence Evaluator	DONE	src/pipeline/confidence.py

Clinical Risk Engine	DONE + trained model	src/clinical_risk/risk_model.py, diabetes_ml_model.joblib
Retinal Structure Segmentation	DONE	src/segmentation/structure_segmentation.py
Telemedicine Simulation	DONE	src/simulation/telemedicine_sim.py
PDF Report Generator	DONE	src/reporting/pdf_generator.py
FHIR Exporter	DONE	src/reporting/fhir_exporter.py
Streamlit Demo	DONE — 1077-line full app	demo/app.py
Unit Tests	DONE	tests/unit/
Integration Tests	DONE	tests/integration/test_pipeline_flow.py
Edge Case Tests	DONE	tests/edge_cases/test_edge_cases.py
A/B Experiment	DONE — results documented	docs/AB_EXPERIMENT_RESULTS.md
Kaggle Training Notebooks	DONE — 4 notebooks	kaggle/N1-N4
MATLAB/Simulink Model	DONE — documented	matlab/
Presentation Materials	DONE — complete	presentation/

MODULE OWNERSHIP

Person	Primary Modules	Key Deliverables
Bezaleel	Pipeline Router, Streamlit UI, Integration	End-to-end flow working, demo functional across platforms
Madhu	Clinical Risk Engine, Retinal Segmentation, Clinical Validation	ICMR thresholds validated, segmentation overlay accurate
Akshay	DR Classifier, Grad-CAM, MATLAB/Simulink	Model trained with QWK metrics, Simulink report generated
Adithya	Tests, A/B Experiments, Edge Cases, Simulation Metrics	150-sample benchmark complete, safety metrics documented

PRESENTATION STRATEGY

- The access gap (2 min)** — 77M+ diabetics, 1 ophthalmologist per 100,000 in rural areas. A distribution problem, not a knowledge problem.
- Why it's stayed unsolved (1 min)** — validated AI screening exists, but hasn't reached low-power, low-connectivity, multilingual, hardware-varied rural settings at scale.

3. **Solution architecture, live (2 min)** — demo run entirely offline/on-device: bad image rejected, good image graded with Grad-CAM, on low-cost hardware.
4. **The evidence (1 min)** — A/B/C experiment results, 0% forced-prediction rate, honest confidence reporting.
5. **The four pillars in numbers (2 min)** — 98.6% bandwidth reduction (sustainability + availability), specialist need cut from 13 to 2 (availability), bilingual reporting with a language roadmap (accessibility), open-source + standards-based export (platform independence).
6. **The compliance path (1 min)** — open-source, offline-ready, hardware-agnostic, ABDM/FHIR-aligned, built with CDSCO/DPDP compliance in mind for real deployment.

What makes it memorable: not a novelty claim, but a working demonstration that the same screening quality validated in hospital pilots can run, unmodified in outcome, on a low-cost camera with no internet connection, in more than one language, on whatever machine the PHC happens to own.

WHAT I NEED FROM EACH OF YOU THIS WEEK

Madhu: - Validate that the clinical risk engine correctly implements ICMR 2024 guidelines - Confirm the segmentation overlay accurately labels OD, Fovea, and vessel tree - Review the bilingual patient guidance text in PDF reports; scope what a third regional language would take - Prepare 2-3 slides on clinical rationale, framed through availability and accessibility

Akshay: - Complete DR classifier training on APTOS 2019, report QWK and per-class sensitivity - Verify Grad-CAM runs in <30 seconds on CPU (no GPU dependency — platform independence) - Prepare MATLAB Simulink model screenshots and the 98.6% bandwidth result - Be ready to explain why EfficientNetB0 over ResNet50/ViT, including the compute-footprint tradeoff

Adithya: - Build the 150-sample benchmark set for A/B/C experiments - Run edge cases: non-fundus images, extreme blur, corruption, adversarial inputs - Document the forced-prediction rate comparison (Baseline vs Our Pipeline) - Compile the safety metrics table for the presentation

I will: - Ensure the Streamlit demo runs flawlessly end-to-end, and confirm it runs across OS/hardware without a fixed dependency on any one platform - Wire all modules together in the pipeline router - Prepare the 8-slide pitch deck, structured around the four pillars - Coordinate the final rehearsal on Day 8

We're not building a demo, and we're not claiming to have invented the underlying detection technique. We're building the complete, honest, open-source system that makes a validated approach actually reach the people who need it — running sustainably on modest hardware, staying available without reliable power or internet, usable by a technician rather than a specialist, and independent of any single platform or vendor. The Streamlit app is just how judges interact with it. The real project is everything that makes it work outside a hospital.

— Bezaleel